Solution: Problem Challenge 1: Search Bitonic Array

Problem Statement

Given a Bitonic array, find if a given 'key' is present in it. An array is considered bitonic if it is monotonically increasing and then monotonically decreasing. Monotonically increasing or decreasing means that for any index i in the array `arr[i] != arr[i+1]`.

Write a function to return the index of the 'key'. If the 'key' appears more than once, return the smaller index. If the 'key' is not present, return -1.

Example 1:

```
Input: [1, 3, 8, 4, 3], key=4
Output: 3
```

Example 2:

```
Input: [3, 8, 3, 1], key=8
Output: 1
```

Example 3:

```
Input: [1, 3, 8, 12], key=12
Output: 3
```

Example 4:

```
Input: [10, 9, 8], key=10
Output: 0
```

Solution

The problem follows the **Binary Search pattern**. Since Binary Search helps us efficiently find a number in a sorted array we can use a modified version of the Binary Search to find the 'key' in the bitonic array.

Here is how we can search in a bitonic array:

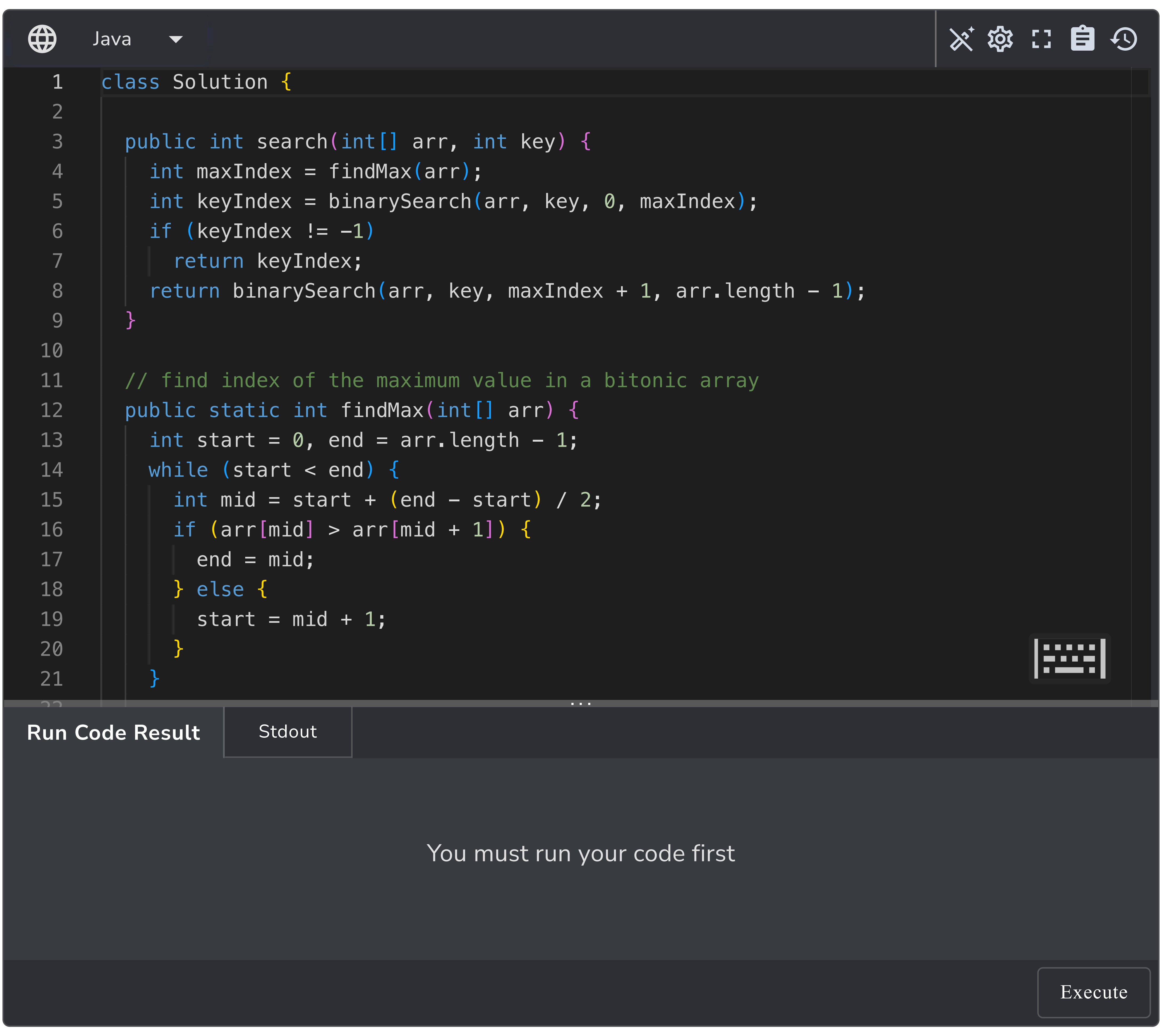
1. First, we can find the index of the maximum value of the bitonic array, similar to `Bitonic Array Maximum`. Let's call the index of the maximum number `maxIndex`.
2. Now, we can break the array into two sub-arrays:

Array from index '0' to `maxIndex`, sorted in ascending order. Array from index `maxIndex+1` to `array_length-1`, sorted in descending order.

3. We can then call **Binary Search** separately in these two arrays to search the 'key'. We can use the same `Order-agnostic Binary Search` for searching.

Code

Here is what our algorithm will look like:



```
class Solution {

  public int search(int[] arr, int key) {
    int maxIndex = findMax(arr);
    int keyIndex = binarySearch(arr, key, 0, maxIndex);
    if (keyIndex != -1)
      return keyIndex;
    return binarySearch(arr, key, maxIndex + 1, arr.length - 1);
  }

  // find index of the maximum value in a bitonic array
  public static int findMax(int[] arr) {
    int start = 0, end = arr.length - 1;
    while (start < end) {
      int mid = start + (end - start) / 2;
      if (arr[mid] > arr[mid + 1]) {
        end = mid;
      } else {
        start = mid + 1;
      }
    }
```

You must run your code first

Time Complexity

Since we are reducing the search range by half at every step, this means that the time complexity of our algorithm will be O(logN) where 'N' is the total elements in the given array.

Space Complexity

The algorithm runs in constant space O(1).